Actions, Assertions, Forms and Test Design

COURSE SPECIFICATIONS

Master interactive browser commands (click, fill, select, check, upload), auto-retrying web-first assertions, comprehensive form testing patterns, and resilient Arrange-Act-Assert test design.

- Track: Playwright Automation with JavaScript + AI • Level: Beginner to Intermediate
- Focus: Actions, Form Controls, Auto-Retrying Assertions • Prerequisites: Days 1–4
- Pattern: AAA (Arrange-Act-Assert) • Test Runner: @playwright/test (.js)

Day 5 Learning Objectives

- Execute core Playwright actions including click(), fill(), clear(), check(), uncheck(), selectOption(), press(), hover(), dblclick(), and setInputFiles().
- Understand Playwright actionability checks: visible, stable, enabled, editable, and receiving events before executing any interaction.
- Write auto-retrying web-first assertions (toBeVisible, toHaveText, toHaveValue, toBeChecked, toHaveURL, toHaveTitle) to prevent flaky timing bugs.
- Distinguish between browser actions and assertions, and explain why one-time state reads (such as page.url() === "...") fail in modern SPAs.
- Automate complex forms combining text fields, dropdowns, checkboxes, radio buttons, validation alerts, and successful submission states.
- Apply the Arrange-Act-Assert (AAA) pattern to structure clear, independent, and maintainable test cases.
- Design positive and negative test scenarios that verify genuine business outcomes rather than superficial button clicks.

In Day 4, you mastered locating elements with semantic, user-facing locators. Now, you will learn how to interact with those elements using Playwright actions.

Core Playwright Actions Overview

Playwright provides dedicated action methods that simulate genuine human user interactions:

```javascript
// 1. Text input
await page.getByLabel('Username').fill('alex.morgan');

// 2. Clear input (resets input value before typing)
await page.getByLabel('Search Query').clear();

// 3. Mouse clicking
await page.getByRole('button', { name: 'Submit' }).click();

// 4. Double clicking
await page.getByRole('button', { name: 'Expand Row' }).dblclick();

// 5. Checkboxes and radio buttons
await page.getByRole('checkbox', { name: 'Accept Terms' }).check();
await page.getByRole('checkbox', { name: 'Subscribe to Newsletter' }).uncheck();

// 6. Select dropdowns (by value, label, or index)
await page.getByLabel('Country').selectOption('US');
await page.getByLabel('Department').selectOption({ label: 'Quality Engineering' });

// 7. Keyboard keys
await page.getByLabel('Search').press('Enter');
await page.getByLabel('Quantity').press('ArrowUp');

// 8. Hovering (useful for dropdown menus and tooltips)
await page.getByRole('navigation').getByText('Solutions').hover();

// 9. File uploads
await page.getByLabel('Upload Resume').setInputFiles('tests/fixtures/sample_resume.pdf');
```

The Actionability Checklist

Before performing any action, Playwright automatically conducts **Actionability Checks**. Unlike legacy tools that throw immediate errors if an element is currently animating or obscured, Playwright pauses and auto-retries until the element satisfies all required criteria:

Action	Visible?	Stable?	Enabled?	Editable?	Receives Events?
`.click()`	Yes	Yes	Yes	-	Yes
`.fill()`	Yes	-	Yes	Yes	-
`.check()`	Yes	Yes	Yes	-	Yes
`.selectOption()`	Yes	-	Yes	-	-
`.hover()`	Yes	Yes	-	-	Yes

Stable means the element has stopped moving or animating (e.g. CSS transitions or slide-in drawers). If a button is disabled, `.click()` automatically waits up to the action timeout for it to become enabled.

Realistic Form Interaction Flow (A complete JavaScript snippet demonstrating standard form controls with accessible locators.)

```
// tests/registration_actions.spec.js
const { test, expect } = require('@playwright/test');

test('completes user profile form using semantic actions', async ({ page }) => {
  await page.goto('https://practice.rootaccess.org/profile-setup');

  // Fill text fields
  await page.getByLabel('Full Name').fill('Sarah Connor');
  await page.getByLabel('Email Address').fill('sarah@cyberdyne.org');

  // Select dropdown option by visible text
  await page.getByLabel('Primary Role').selectOption({ label: 'Senior Automation QA' });

  // Toggle radio and checkboxes
  await page.getByRole('radio', { name: 'Full-Time Employment' }).check();
  await page.getByRole('checkbox', { name: 'Subscribe to Security Bulletins' }).check();

  // Upload portfolio avatar
  await page.getByLabel('Profile Avatar').setInputFiles('fixtures/avatar.png');

  // Submit
```

Writing bulletproof test assertions is what separates an automation professional from an amateur. In Playwright, assertions are **web-first**.

The Problem with One-Time Reads

In legacy frameworks or poorly written tests, engineers often write:
```javascript
// BAD PRACTICE: One-time read!
const isVisible = await page.getByRole('alert').isVisible();
expect(isVisible).toBe(true); // Fails instantly if the alert takes 200ms to animate in!
```

Why does this fail?
1. `await page.getByRole('alert').isVisible()` queries the DOM at that precise microsecond.
2. If your server is processing the request and the banner takes 300ms to appear, `isVisible()` returns `false` immediately.
3. The generic Jest/Chai assertion `expect(false).toBe(true)` throws an uninformative test failure.

The Solution: Web-First Assertions

Playwright provides web-first assertions via the `expect(locator)` syntax. These assertions do **NOT** read once; they **smart-poll** the browser continuously until the condition passes or the assertion timeout (default 5,000ms) expires:

```javascript
// GOOD PRACTICE: Web-first assertion with smart auto-retrying!
await expect(page.getByRole('alert')).toBeVisible();
```

Essential Web-First Assertions

Assertion	Verifies
`await expect(locator).toBeVisible()`	Element is visible in the viewport and not display:none
`await expect(locator).toBeHidden()`	Element is removed from DOM or visually hidden
`await expect(locator).toHaveText('Welcome')`	Element text strictly matches expected string or RegExp
`await expect(locator).toContainText('alex')`	Element contains expected substring
`await expect(locator).toHaveValue('qa-lead')`	Input / select value attribute matches
`await expect(locator).toBeEnabled()`	Element is interactive and not disabled
`await expect(locator).toBeDisabled()`	Element has disabled attribute or aria-disabled

`await expect(locator).toBeChecked()`	Checkbox or radio button is checked
`await expect(locator).toHaveCount(3)`	Locator resolves to exactly 3 matching elements
`await expect(page).toHaveURL(/.*dashboard/)`	Browser URL matches string or regular expression
`await expect(page).toHaveTitle(/.*RootAccess/)`	Page document title matches expected pattern

Inverting Assertions (.not)

Every web-first assertion can be inverted with `.not`:

```javascript

// Verify submit button is disabled while form is invalid:

await expect(page.getByRole('button', { name: 'Submit' })).toBeDisabled();

// Verify loader disappears once table data is populated:

await expect(page.getByTestId('loading-spinner')).not.toBeVisible();

```

**Comparing One-Time vs Web-First Assertions (A direct side-by-side comparison illustrating why web-first assertions eliminate test flakiness.)**

```
// 'L Flaky: Will fail if API takes 150ms to return notification:
const text = await page.getByRole('alert').innerText();
expect(text).toContain('Account created');

// ' Bulletproof: Auto-retries every 50ms until banner arrives or 5s elapses:
await expect(page.getByRole('alert')).toContainText('Account created');
```

Forms are the foundation of business web applications—from sign-up workflows and checkout carts to banking wire transfers.

### Structuring Form Test Cases

A robust form automation suite always exercises both **Happy Path** (positive) and **Boundary / Negative** validations:

1. **Required Field Validation (Negative):** Submitting with empty inputs must display required field error indicators.
2. **Format Validation (Negative):** Submitting invalid emails (e.g. `plainaddress`) or weak passwords triggers specific inline error messages.
3. **Valid Submission (Positive):** Filling all inputs correctly submits the form and displays a clear success banner or redirects to the dashboard.

### Best Practices for Form Elements

```javascript
// 1. Text Inputs: Target by accessible label
await page.getByLabel('First Name').fill('Jessica');

// 2. Passwords: Target by label (keeps passwords out of selectors)
await page.getByLabel('Password', { exact: true }).fill('SuperSecret123!');

// 3. Dropdowns: Verify selection with toHaveValue()
const roleDropdown = page.getByLabel('Job Level');
await roleDropdown.selectOption('lead');
await expect(roleDropdown).toHaveValue('lead');

// 4. Radio Groups: Check the specific option
await page.getByRole('radio', { name: 'Annual Billing (Save 20%)' }).check();
await expect(page.getByRole('radio', { name: 'Annual Billing (Save 20%)' })).toBeChecked();

// 5. Submit Button: Verify disabled state, then click
const submitBtn = page.getByRole('button', { name: 'Create Workspace' });
await expect(submitBtn).toBeEnabled();
await submitBtn.click();
```

Asserting Form Validation States

When testing validation, verify:

1. That the specific error message is visible:
   `await expect(page.getByText('Valid corporate email required')).toBeVisible();`
2. That the form did NOT navigate away:
   `await expect(page).toHaveURL(/.*register/);`
3. That the submit button was not submitted multiple times.
```

Comprehensive Form Automation Test (Testing required field errors followed by valid submission.)

```
test('validates required fields before allowing submission', async ({ page }) => {
  await page.goto('/register');

  // Attempt submitting empty form
  await page.getByRole('button', { name: 'Sign Up' }).click();

  // Verify inline validation errors
  await expect(page.getByText('Name is required')).toBeVisible();
  await expect(page.getByText('Email is required')).toBeVisible();

  // Now fill required inputs
  await page.getByLabel('Full Name').fill('David Miller');
  await page.getByLabel('Email Address').fill('david@rootaccess.org');
  await page.getByLabel('Password').fill('ValidPass123!');
  await page.getByRole('checkbox', { name: 'I agree to the terms' }).check();

  // Submit and verify success confirmation
  await page.getByRole('button', { name: 'Sign Up' }).click();
  await expect(page.getByRole('heading', { name: 'Welcome to RootAccess!' })).toBeVisible();
});
```

> Writing automated tests is software engineering. A test that is hard to read or reason about creates technical debt for the entire engineering team.

The Arrange-Act-Assert (AAA) Pattern

Every test case should cleanly separate into three distinct phases:

1. **Arrange:** Set up preconditions, navigate to the target URL, configure mock data or seed states.
2. **Act:** Perform the user actions under test (type into fields, click buttons, select items).
3. **Assert:** Verify the expected outcomes using web-first assertions.

```javascript
test('subscribes user to newsletter with valid email', async ({ page }) => {
  // 1. ARRANGE
  await page.goto('/newsletter');
  const emailInput = page.getByPlaceholder('Enter your email');
  const subscribeButton = page.getByRole('button', { name: 'Subscribe' });

  // 2. ACT
  await emailInput.fill('developer@rootaccess.org');
  await subscribeButton.click();

  // 3. ASSERT
  await expect(page.getByRole('alert')).toHaveText('Thank you for subscribing!');
  await expect(emailInput).toHaveValue(''); // input cleared after submit
});
```

Test Independence (No Shared State)
- Each test must run independently from every other test.
- Test B must **never rely on Test A having logged in or created a database record.**
- If tests share mutable state, running them in parallel (Playwright's default) causes intermittent, unexplained failures.

Meaningful Test Names
- 'L Bad: `test('test 1', async ({ page }) => { ... });`
- 'L Bad: `test('form works', async ({ page }) => { ... });`
- ' Good: `test('should show password mismatch error when passwords do not match', async ({ page }) => { ... });`
- ' Good: `test('should navigate to dashboard upon entering valid credentials', async ({ page }) => { ... });`

Asserting Outcomes vs Clicking Buttons
An automation script that clicks 10 buttons without asserting any outcome is not a test—it is merely an automated monkey. Always assert the observable business result (e.g. success banner, updated badge, modal closed, redirected URL).

Clean AAA Test Structure (A well-structured test with distinct Arrange, Act, and Assert phases.)

```
test('should display validation error when registration email is invalid', async ({ page }) => {
  // Arrange
  await page.goto('/signup');
  const emailInput = page.getByLabel('Email Address');
  const submitButton = page.getByRole('button', { name: 'Continue' });

  // Act
  await emailInput.fill('invalid-email-format');
  await submitButton.click();

  // Assert
  await expect(page.getByText('Please enter a valid email address')).toBeVisible();
  await expect(submitButton).toBeEnabled();
});
```

In this practical lab, you will design and implement a comprehensive test suite for a realistic user registration workflow.

The Simulated Target Application

The application under test is a SaaS developer workspace registration form at `/app/register` with the following specification:

```html
<!-- Registration Form DOM Structure -->
<form id="registration-form">
  <label for="name-input">Full Name *</label>
  <input id="name-input" type="text" placeholder="e.g. Alex Morgan" required />

  <label for="email-input">Work Email *</label>
  <input id="email-input" type="email" placeholder="alex@company.com" required />

  <label for="role-select">Primary Engineering Role *</label>
  <select id="role-select">
    <option value="">Select a role...</option>
    <option value="qa_lead">Lead QA Automation Engineer</option>
    <option value="sdet">Software Development Engineer in Test (SDET)</option>
    <option value="devops">DevOps / CI-CD Specialist</option>
  </select>

  <label>
    <input type="checkbox" name="terms" /> I agree to the RootAccess Terms of Service *
  </label>

  <button type="submit">Complete Registration</button>
</form>

<!-- Validation Alert -->
<div role="alert" class="error-banner" style="display:none;"></div>

<!-- Confirmation Screen -->
<div class="registration-success" style="display:none;">
  <h2>Registration Confirmed!</h2>
  <p>Welcome aboard, <span id="user-display"></span>.</p>
</div>
```

Lab Requirements (5 Test Cases):
1. **Test 1: Required Fields Validation: Submitting an empty form shows the error alert "All required fields must be completed".**
2. **Test 2: Invalid Email Validation: Entering `not-an-email` triggers "Please enter a valid work email".**
3. **Test 3: Terms Checkbox Enforcement: Filling valid name and email but leaving terms unchecked blocks submission.**
4. **Test 4: Dropdown Selection: Selecting 'sdet' correctly updates the dropdown value.**
5. **Test 5: Full Successful Registration (Happy Path): Filling all fields, selecting role, checking terms, and clicking submit displays "Registration Confirmed!".**

Reference Test Case Structure (A sample test case demonstrating the expected structure for the lab.)

```
test('should show error when email is malformed', async ({ page }) => {
  // Arrange
  await page.goto('/app/register');

  // Act
  await page.getByLabel('Full Name').fill('Marcus Wright');
  await page.getByLabel('Work Email').fill('marcus-invalid-email');
  await page.getByLabel('Primary Engineering Role').selectOption('qa_lead');
  await page.getByRole('checkbox', { name: /Terms of Service/i }).check();
  await page.getByRole('button', { name: 'Complete Registration' }).click();

  // Assert
  await expect(page.getByRole('alert')).toContainText('Please enter a valid work email');
});
```

and checkboxes, apply at least 4 auto-retrying web-first assertions, configure a custom timeout, test negative error validations, and structure with the AAA pattern.

Task: Submit tests/day5-registration-form.spec.js with complete terminal logs and technical actionability explanations.

Assignment Starter Code (tests/day5-registration-form.spec.js)

```
// tests/registration_form.spec.js
// DAY 5 ASSIGNMENT: Actions, Assertions, Forms and Test Design
// Complete the test suite below following the Arrange-Act-Assert (AAA) pattern.

const { test, expect } = require('@playwright/test');

test.describe('User Registration Form Suite', () => {

  test('TC-1: should show required field validation when submitting empty form', async ({ page }) => {
    // 1. Arrange: Navigate to registration form
    await page.goto('/app/register');

    // 2. Act: Click submit without filling required fields
    await page.getByRole('button', { name: 'Complete Registration' }).click();

    // 3. Assert: Verify required fields error banner appears
    await expect(page.getByRole('alert')).toBeVisible();
    await expect(page.getByRole('alert')).toContainText('All required fields must be completed');
  });

  test('TC-2: should show error when email format is invalid', async ({ page }) => {
```

10 Rubric Requirements for Day 5 Assignment:

1. Pure JavaScript structure using test and expect from @playwright/test
2. Fills multiple text inputs using locator.fill()
3. Selects dropdown options via locator.selectOption()
4. Selects radio buttons and checks/unchecks checkboxes using locator.check() / click()
5. Uses at least 4 web-first assertions (toBeVisible, toHaveValue, toBeChecked, etc.)
6. Includes custom assertion timeout { timeout: 8000 } for delayed feedback
7. Validates negative state (.not.toBeVisible or error message banner)
8. Cleanly structured following AAA (Arrange-Act-Assert) pattern
9. Complete terminal test execution log showing passed status
10. Comprehensive technical explanation of actionability checks and web-first design

Q1. What is the difference between an action and an assertion in Playwright?

[Difficulty: Beginner] Evaluation Focus: Understanding core browser automation concepts and separation of concerns.

Suggested Answer: An action (e.g. click(), fill(), selectOption()) instructs Playwright to interact with an element and simulate human behavior. Playwright auto-waits for actionability checks before executing the action. An assertion (e.g. expect(locator).toBeVisible(), toHaveText()) verifies that the application is in an expected state. Web-first assertions smart-poll the browser continuously until the condition evaluates to true or times out.

Follow-Up Question: "What happens if an action fails its actionability checks versus when an assertion fails?"

Q2. Why does Playwright recommend web-first assertions over reading a property and checking it with Jest/Chai assertions?

[Difficulty: Intermediate] Evaluation Focus: Flakiness prevention, asynchronous rendering, and smart polling.

Suggested Answer: Reading a property directly (e.g. const val = await page.getByRole("alert").textContent(); expect(val).toBe("Saved");) performs an immediate, one-time read of the DOM. If the application is asynchronous and renders the alert 100ms later, the one-time read immediately fails. Web-first assertions (await expect(locator).toHaveText("Saved")) continuously poll the element in the browser until it matches or reaches the assertion timeout, completely eliminating race conditions.

Follow-Up Question: "How can you configure the default timeout for web-first assertions globally?"

Q3. How do you automate a file upload in Playwright, and why is setInputFiles() preferred over clicking the upload button?

[Difficulty: Intermediate] Evaluation Focus: Handling native operating system file dialogs without external tools.

Suggested Answer: Playwright uses locator.setInputFiles("path/to/file.pdf") to set files directly onto an <input type="file"> element. Clicking a file upload button would launch a native OS file picker window (Finder on macOS or Explorer on Windows), which browser automation protocols cannot interact with. setInputFiles() bypasses the native dialog safely and triggers appropriate change and input JavaScript events on the element.

Follow-Up Question: "How would you handle a custom drag-and-drop file upload zone that lacks a visible file input?"

Q4. You have a submit button that remains disabled until all 4 form fields are valid. How would you test this behavior with Playwright?

[Difficulty: Scenario-Based] Evaluation Focus: Dynamic form state validation and negative testing.

Suggested Answer: First, navigate to the form and assert that the submit button is initially disabled using await expect(submitBtn).toBeDisabled(). Next, fill fields 1, 2, and 3 one by one, asserting after each that the button remains disabled. Finally, fill the 4th required field and assert await expect(submitBtn).toBeEnabled(). Then click the button and assert the successful form submission outcome.

Follow-Up Question: "What assertion would you use to verify that an input field border turned red indicating an error state?"

Q5. Explain the Arrange-Act-Assert (AAA) pattern and why test independence is critical when executing Playwright in parallel.

[Difficulty: Intermediate] Evaluation Focus: Test architecture, maintainability, and parallel execution.

Suggested Answer: The Arrange-Act-Assert pattern standardizes test structure: Arrange prepares test data and navigation, Act performs user operations, and Assert validates the resulting state. Test independence means each test case is completely self-contained and creates its own preconditions. In Playwright, tests run in parallel across separate worker processes by default. If Test B depends on cookies or database entities created by Test A, parallel execution will cause intermittent failures due to non-deterministic test order.

Follow-Up Question: "What hooks or fixtures does Playwright offer to help with the Arrange phase?"

1. Which method should you use to enter text into an input field, automatically clearing any previous text first?
Key Concept: locator.fill() is the recommended Playwright action. It focuses the input, clears existing text, and fills the new string atomically.

2. What is the key difference between page.getByRole("alert").isVisible() and expect(page.getByRole("alert")).toBeVisible()?
Key Concept: isVisible() does not poll—it returns a boolean snapshot at that millisecond. expect().toBeVisible() is web-first and polls until the element appears.

3. Which assertion correctly verifies that an input field currently contains the value "alex@rootaccess.org"?
Key Concept: Inputs hold their contents in the DOM "value" property, which is asserted with toHaveValue(). toHaveText() inspects inner text, which is empty for self-closing input elements.

4. How do you select an option with visible text "Lead Automation QA" from a native HTML <select> dropdown?
Key Concept: selectOption({ label: "..." }) selects options by their human-visible text in standard <select> dropdowns.

5. How does Playwright handle file uploads for <input type="file"> elements?
Key Concept: locator.setInputFiles() sets file references directly on the file input, avoiding native OS file dialogs completely.

6. Which assertion confirms that a checkbox is currently selected?
Key Concept: toBeChecked() is the web-first assertion to verify that checkboxes or radio buttons are in a checked state.

7. In the Arrange-Act-Assert (AAA) pattern, what belongs in the "Arrange" phase?
Key Concept: The Arrange phase sets up everything needed for the test to execute cleanly before any interaction occurs.

8. What is the default timeout for web-first assertions in Playwright unless overridden?
Key Concept: Playwright web-first assertions have a default timeout of 5,000ms (5 seconds), whereas general test actions typically default to 30,000ms.

9. How do you assert that a loading spinner has disappeared from the screen after an API call completes?
Key Concept: Using .not.toBeVisible() or .toBeHidden() polls until the element is either removed from the DOM or hidden via CSS display/visibility.

10. Why is it an anti-pattern to make tests dependent on the execution results of previous tests?
Key Concept: Test independence ensures tests can execute concurrently across CPU cores and in any order without failing due to missing shared state.